

三节点混合部署最佳实践

<https://github.com/pingcap/docs-cn/blob/95e8751594c2c4881d60948342f81ecd25fb9382/best-practices/three-nodes-hybrid-deployment.md>

三节点混合部署的最佳实践

在对性能要求不高且需要控制成本的场景下，将 TiDB、TiKV、PD 混合部署在三台机器上是一个可行的方案。

本文以 TPC-C 作为工作负载，提供一些在三节点混合部署场景下部署和参数调整的建议。

部署环境和测试方法

三台物理机，每台机器有 16 个 CPU 核心且内存为 32 GB。每节点（台机器）上混合部署 1 TiDB（实例）+ 1 TiKV（实例）+ 1 PD（实例）。

由于 PD 和 TiKV 都会存储信息到磁盘，磁盘的写入读取延迟会直接影响到 PD 和 TiKV 的服务延迟，为了防止 PD 和 TiKV 对磁盘资源的争夺导致相互影响，推荐 PD 和 TiKV 采用不同的磁盘。

在 TiUP bench 中使用 TPC-C 5000 Warehouse 数据，每次以 terminals 参数为 128 的并发度测试 12 小时。主要观察集群性能的稳定性指标。

下图是默认参数配置下，12 小时内集群的 QPS 监控，可以看倒有比较明显的抖动。



QPS with default config

调整参数后，稳定性得到了改善。



QPS with modified config

参数调整

上图中出现抖动的主要原因是，默认的线程池和后台任务资源分配针对的是资源比较充足的机器。在混合部署的场景下，因为可用资源需要被多个组件共享，所以需要通过配置参数进行限制。

本次测试最终采用的配置如下：

tikv:

```
readpool.unified.max-thread-count: 6
server.grpc-concurrency: 2
storage.scheduler-worker-pool-size: 2
gc.max-write-bytes-per-sec: 300K
rocksdb.max-background-jobs: 3
rocksdb.max-sub-compactions: 1
rocksdb.rate-bytes-per-sec: "200M"
```

```
tidb:
performance.max-procs: 8
```

接下来会分别介绍这几个参数的意义和调整方法。

TiKV 线程池大小配置

本节的配置项会调整一些与前台业务有关的线程池的资源配比。缩减这些线程池会损失一些性能，但在混合部署场景下可用资源有限，本身也难以达到很高的性能，所以选择牺牲一小部分性能去换取整体的稳定性。

可以先以默认配置为基础运行一次实际负载的测试，观察各线程池的实际使用量，再修改这些配置项，缩减利用率不高的线程池大小。

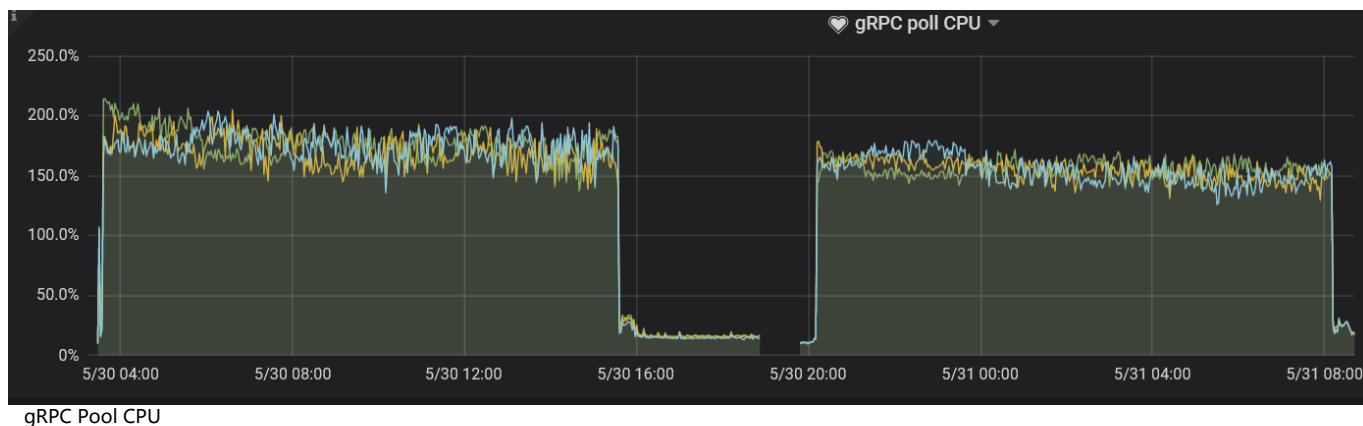
readpool.unified.max-thread-count

该参数默认取值为机器线程数的 80%，因为是混合部署场景，你需要手动计算并指定该值。可以先设置成期望 TiKV 使用的 CPU 线程数的 80%。

server.grpc-concurrency

因为现有部署方案的 CPU 资源有限，实际请求数也不会很多，可以把 `server.grpc-concurrency` 参数值调低，后续观察监控面板保持其使用率在 80% 以下即可。

本次测试最后选择设置该参数值为 2，通过 gRPC poll CPU 面板观察，利用率正好在 80% 左右。



storage.scheduler-worker-pool-size

在 TiKV 检测到机器 CPU 数大于或等于 16 时，该参数值默认为 8。CPU 数小于 16 时，该参数值默认为 4。它主要用于将复杂的事务请求转化为简单的 key-value 读写。但是 scheduler 线程池本身不进行任何写操作。

一般来说该线程池的利用率保持在 50% - 75% 之间是比较好的。和 gRPC 线程池情况类似，混合部署时该参数默认取值偏大，资源利用不充分。本次测试最后选择取值为 2，通过 Scheduler worker CPU 面板观察，利用率比较符合最佳实践。